Opportunity Hunter AI

Complete Beginner's Learning Guide & Technical Interview Masterclass

Autonomous Browser AI Agent • n8n Automation • FastAPI • React 18 • SQLite

Project Mission: *Opportunity Hunter AI is a production-quality, portfolio-grade system that empowers students and early-career developers to discover internships, hackathons, and research roles. It eliminates manual searching through autonomous browser navigation, zero-hallucination data extraction, smart URL normalization & deduplication, and automated scheduled workflows.*

Chapter 1: Introduction to AI Agents & The Problem We Solve

1.1 What is an AI Agent? (From Code to Agents)

To understand Opportunity Hunter AI, we must first understand how artificial intelligence has evolved from deterministic code to Large Language Models (LLMs) and finally to **Autonomous AI Agents**:

Paradigm	How It Works	Strengths & Weaknesses
Traditional Program (Deterministic Code)	Executes hardcoded if/else logic and static DOM selectors (e.g., XPath, BeautifulSoup).	Fast and predictable, but breaks instantly when websites change HTML class names or layout.
Standard LLM (Knowledge Advisor)	Generates conversational answers based on static pre-training weights (ChatGPT, Claude).	Understands complex context, but has no live web access, cannot take actions, and hallucinates.
Autonomous AI Agent (The Digital Intern)	Combines LLM reasoning with external tools (Headless Browser) in a Perception-Action loop.	Visually perceives webpages, interacts with buttons/filters, extracts structured data, and self-heals.

1.2 The ReAct (Reason + Act) Loop Explained

Opportunity Hunter AI does not simply 'scrape' pages; it operates on the **ReAct loop**:

- **1. Perceive:** The agent inspects the Chromium DOM tree and visual accessibility tree, assigning numbers to interactive elements.
- **2. Reason:** The agent feeds the DOM state and current goal to the LLM: *'Find the search bar and filter for software internships'*.
- **3. Act:** The LLM emits a structured action (e.g., click_element(index=14) or input_text(index=8, text='AI')).
- **4. Evaluate:** The agent observes the resulting state. If an anti-bot modal appears, it closes it; if results load, it extracts them.

1.3 The Real-World Problem Solved

Students waste 10 to 15 hours every week manually checking 20+ fragmented portals (LinkedIn, Wellfound, Devpost, Y Combinator, university career boards). They constantly encounter expired roles, duplicate reposts, and missed rolling deadlines. **Opportunity Hunter AI** completely automates this workflow: searching dynamically, extracting verified deadlines, eliminating duplicate postings, and notifying students automatically.

1.4 High-Level System Architecture

```
+-------------------------------------------------------------------------+ | FRONTEND LAYER | | React 18 + Vite + Tailwind CSS + Lucide Icons + React Router (Port 5173)| | * Dashboard Page * Discover Page (Live Stepper) * Results Page | | * Saved / Applied Tracker * Settings & Schedule Configurator |
+-----------------------------------+-----------------------------------+ | HTTP REST / JSON v
+-------------------------------------------------------------------------+ | BACKEND LAYER | | FastAPI + Uvicorn + Pydantic v2 (Port 8000) | | * REST Endpoints (/api/research, /api/opportunities, /api/preferences) | | * Background Task Manager (Async execution & run state tracking) | | * Deduplication Engine (URL normalization & non-destructive merging) | | * SQLite Database (Persistent relational store with indexed queries) |
+-----------------+-----------------------------------+------------------+ | | v (Async Background Tasks) v (Webhooks / HTTP) +-----------------------------------+ +---------------------------------+ | AGENTIC ENGINE | | AUTOMATION ENGINE | | Browser Use Web AI + Playwright | | n8n Workflow Hub | | * Chromium Headless Browser | | * Workflow 1: Manual Run | | * DOM Tree Analyzer & Controller | | * Workflow 2: Daily Cron (8 AM) | | * LLM: OpenRouter / OpenAI / Mock | | * Workflow 3: HTML Email Digest | | * Pydantic Schema Output Validator | | * Webhook Callbacks into FastAPI | +-----------------------------------+ +---------------------------------+
```

Chapter 2: Comprehensive Technology Stack Deep-Dive

Technology	Exact Role in Opportunity Hunter AI	Engineering Rationale & Trade-offs
Python 3.12+	Core programming language for backend, services, and agent execution.	Native ecosystem for modern AI agent libraries; enhanced execution performance in 3.12+.
FastAPI	Asynchronous REST API framework (Port 8000), CORS, Swagger docs.	Selected over Flask/Django for high-concurrency async support, Pydantic validation, and OpenAPI generation.
SQLite	Embedded ACID relational database storing opportunities, runs, and preferences.	Zero-configuration local database requiring no Docker or cloud setup; indexed on URL, type, and status.
Browser Use Web AI	Agent framework enabling LLMs to navigate websites and interact with DOM nodes.	Translates visual DOM elements into simplified accessibility trees; handles anti-bot popups and clicks.
Playwright Chromium	Headless browser automation engine controlled by Browser Use.	Talks directly to Chrome DevTools Protocol (CDP); far more reliable than legacy Selenium WebDriver.
n8n Automation	Visual workflow engine managing cron schedules, manual runs, and HTML emails.	Decouples scheduling and alerting from FastAPI; enables operators to adjust workflows visually.
React 18 & Vite	Single-page SaaS dashboard (Port 5173) with live polling stepper.	Vite provides instant HMR development (<300ms) and optimized production bundles compared to CRA.
Tailwind CSS	Utility-first styling framework across all pages and components.	Eliminates monolithic CSS style sheets; enables responsive design directly inside JSX templates.
Pydantic v2	Strict data validation, type coercion, and LLM schema definitions.	Rust-backed performance; guarantees structured JSON outputs and prevents LLM date hallucinations.

Chapter 3: Step-by-Step Implementation & Code Walkthrough

3.1 Folder Structure & Modular Architecture

```
opp-hunter/ ■■■ backend/app/ ■ ■■■ main.py # FastAPI entrypoint, CORS, lifespan ■ ■■■ agents/browser_researcher.py # Browser Use AI agent & Mock engine ■ ■■■ api/endpoints.py # REST API endpoints & Webhooks ■ ■■■ database/db.py # SQLite schema, indexes, and CRUD ■ ■■■ models/opportunity.py # Pydantic v2 data models & schemas ■ ■■■ services/deduplication.py # URL normalizer & deduplication engine ■■■ frontend/src/ ■ ■■■ pages/DiscoverPage.jsx # Research form & 4-step polling stepper ■ ■■■ pages/ResultsPage.jsx # Filterable opportunity grid & status selector ■ ■■■ services/api.js # HTTP REST client wrapper ■■■ n8n/workflows/ # Pre-configured manual & scheduled workflows ■■■ docs/ # Comprehensive documentation library
```

3.2 The Strict Pydantic Data Model (`models/opportunity.py`)

By typing deadline: Optional[str] = None and stipend: Optional[str] = None, we programmatically enforce our **Zero-Hallucination Policy**. If a website does not clearly post a deadline, it outputs null rather than allowing the LLM to invent a plausible date:

```
class OpportunityBase(BaseModel): title: str = Field(..., description='Opportunity title') company: str = Field(..., description='Company or organizing body') location: str = Field(default='Remote', description='Physical location or Remote') type: OpportunityType = Field(default=OpportunityType.INTERNSHIP) url: str = Field(..., description='Direct application link') deadline: Optional[str] = Field(default=None, description='ISO date or null') stipend: Optional[str] = Field(default=None) match_score: float = Field(default=0.0, ge=0.0, le=100.0) status: OpportunityStatus = Field(default=OpportunityStatus.NEW)
```

3.3 Smart URL Normalization & Status Preservation Guarantee

Web crawlers frequently encounter identical listings with different marketing queries (`utm_*`, `ref`, `fbclid`). Our deduplication engine normalizes URLs before querying SQLite. Crucially, it provides a **Status Preservation Guarantee**: if a student has marked an opportunity as applied, re-crawling the web will refresh the opportunity's timestamp but will NEVER overwrite the user's status back to new.

```
def process_and_deduplicate(opportunities: list[OpportunityBase]) -> tuple[int, int]: added, updated = 0, 0 for opp in opportunities: clean_url = normalize_url(opp.url) opp.url = clean_url existing = get_opportunity_by_url(clean_url) if existing: # STATUS PRESERVATION GUARANTEE: keep user status (e.g. 'applied') opp.status = existing.status update_opportunity(existing.id, opp) updated += 1 else: if not opp.status: opp.status = OpportunityStatus.NEW insert_opportunity(opp) added += 1 return added, updated
```

3.4 Asynchronous API & Polling Endpoint (`api/endpoints.py`)

Because autonomous browser navigation can take 30 to 60 seconds, standard synchronous HTTP requests would hit client timeouts. We implemented the **HTTP 202 Accepted** asynchronous job pattern:

```
@router.post('/api/research', status_code=status.HTTP_202_ACCEPTED) async def start_research(request: ResearchRequest, background_tasks: BackgroundTasks): run_id = str(uuid.uuid4()) create_research_run(run_id=run_id, query=request.query) # Dispatches agent execution in background; returns HTTP 202 in < 20ms background_tasks.add_task(execute_research_job, run_id, request) return {'status': 'started', 'run_id': run_id} @router.get('/api/research/{run_id}') async def get_run_status(run_id: str): # Polled by React frontend every 2 seconds to update live 4-step UI stepper return get_research_run(run_id)
```

3.5 Local Setup Commands

```
# 1. Setup Backend Virtual Environment & Dependencies cd "/Users/saumyapant/Desktop/opp hunter" python3 -m venv .venv && source .venv/bin/activate pip install -r backend/requirements.txt cp backend/.env.example backend/.env # 2. Launch FastAPI Backend python3 -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000 --reload # 3. Launch React Frontend cd frontend && npm install && npm run dev # 4. Run Test Suite (13 tests) python3 -m unittest discover -s backend/tests -p "test_*.py" -v
```

Chapter 4: The Agent Under the Hood — Execution Lifecycle

4.1 The 4-Step Discovery Sequence

Step	Name	System Actions & Data Transformations
Step 1	**Initializing**	FastAPI generates UUID `run_id`, records run in SQLite, and launches async background task.
Step 2	**Navigating**	Playwright Chromium boots headless; navigates opportunity boards; extracts interactive accessibility DOM tree.
Step 3	**Extracting**	LLM reasons over DOM nodes; extracts structured listings into Pydantic models; missing deadlines coerced to null.
Step 4	**Deduplicating**	Tracking query params stripped; URLs normalized; SQLite upsert executed with Status Preservation; final summary returned.

Chapter 5: Technical Interview Masterclass & Defense Preparation

5.1 Project Pitching Scripts

The 1-Minute Recruiter Pitch:

"Opportunity Hunter AI is an intelligent opportunity aggregator that solves job search fatigue. Traditional web scrapers break whenever a website updates its HTML class names. To solve this, I used Browser Use and Playwright, allowing an LLM to visually perceive web pages and interact with them like a human. I built the backend in FastAPI with SQLite, ensuring high-concurrency job dispatching via asynchronous workers and polling endpoints. I also implemented a robust deduplication engine that normalizes URLs and ensures that if a student marks a role as 'Applied', a subsequent crawl will never overwrite that status. Finally, I built a sleek React SaaS interface featuring a live 4-step progress stepper and n8n orchestration for scheduled notifications."

5.2 Top 10 Technical Interview Questions & Model Answers

Q1: Why did you choose Browser Use over traditional web scraping libraries like BeautifulSoup or Scrapy?

BeautifulSoup and Scrapy are static HTML parsers that make a GET request and parse raw HTML strings. However, 90% of modern opportunity boards—like LinkedIn, Wellfound, and Greenhouse—rely heavily on client-side JavaScript, infinite scrolling, and interactive modals. Furthermore, traditional scrapers rely on hardcoded CSS selectors. If the website changes a class name from .job-card to .listing-container, the scraper breaks immediately. Browser Use controls a real headless Chromium browser via Playwright. It converts the visual DOM into interactive accessibility elements and allows an LLM to reason about what to click, scroll, or fill out. This makes our crawler self-healing and capable of navigating complex user flows.

Q2: What is an asynchronous API, and why did you use the HTTP 202 Accepted pattern?

A standard synchronous API keeps the HTTP connection open until the server finishes processing. If an AI agent takes 45 seconds to open a browser, navigate pages, and extract data, a standard synchronous request would either hit a browser timeout or block incoming requests on a single-threaded server. To prevent this, I implemented the asynchronous job pattern: When the client calls POST /api/research, FastAPI assigns a unique UUID, records the job status as 'initializing' in SQLite, and dispatches the task to BackgroundTasks. FastAPI immediately returns HTTP 202 Accepted with the run_id in less than 20ms. The React frontend then periodically polls GET /api/research/{run_id} to retrieve progress updates, keeping the UI responsive.

Q3: How do you prevent the LLM from hallucinating deadlines or stipends?

LLMs are generative models naturally inclined to produce plausible-sounding completions when facts are missing. In job hunting, inventing a deadline like 'May 1st' when none exists is disastrous. I solved this at both the schema and prompt levels: In our Pydantic model, deadline and stipend are typed as Optional[str] = None. In our system prompt, we explicitly instruct the LLM: 'Extract only information explicitly stated on the webpage. If a deadline or stipend is not explicitly mentioned, you MUST set the field to null. NEVER guess, estimate, or extrapolate dates.' Pydantic validates the parsed JSON against this contract.

Q4: Explain how your URL normalization and deduplication works.

A major challenge with web crawlers is that the same job listing might appear under multiple different URLs due to marketing queries. For example: 'company.com/jobs/123?utm_source=linkedin' and 'company.com/jobs/123/' are the exact same role. My deduplication pipeline normalizes URLs by: lowercasing the domain, standardizing HTTPS, stripping default ports (:80, :443), stripping tracking query parameters (utm_*, ref, source, fbclid, gclid), and removing redundant trailing slashes. When matching against SQLite, if a record exists, we update the timestamp but preserve the user's status (applied, saved).

Q5: What is the Status Preservation Guarantee, and why was it necessary?

During early testing, I discovered a subtle data-clobbering bug: if a user discovered an internship, saved it, and later marked it as 'Applied', and then ran a new search the following week, the crawler would discover the listing again. Because default schemas assign status = 'new', re-inserting the listing would overwrite the user's 'Applied' status back to 'new'. To fix this, I introduced the Status Preservation Guarantee in our service layer: before writing an opportunity to SQLite, we query by normalized URL. If the listing already exists, we copy the user's existing status to the incoming model before executing the SQL UPDATE.

Q6: Why did you use both FastAPI and n8n instead of writing everything in Python?

This is an architectural separation of concerns. FastAPI is our core domain engine: it handles database integrity, REST API endpoints, business logic, and browser automation. n8n is our workflow orchestrator: it specializes in event-driven integrations, cron scheduling, visual pipeline monitoring, and external alerting (SMTP emails, Discord webhooks). If we put cron schedules and email templates inside FastAPI, our web server would become tightly coupled to external email providers. With n8n, an operator can adjust schedules and inspect payload history visually without touching Python code.

Q7: How does your frontend communicate with the backend?

The frontend communicates with the backend via standard HTTP REST over JSON using our centralized api.js client. For long-running research tasks, we use an asynchronous polling architecture: 1. POST /api/research triggers the job and returns a run_id. 2. The React component uses a JavaScript setInterval to call GET /api/research/{run_id} every 2000 milliseconds. 3. Each poll returns the current execution state (step, status, logs). The frontend translates this into a 4-step animated UI stepper. 4. When the response status changes to completed or failed, the interval is cleared with clearInterval to prevent memory leaks.

Q8: How would you scale this application to 100,000 active users?

To scale to 100k users, I would implement three architectural changes: 1. Database Migration: Migrate SQLite to a managed PostgreSQL cluster with read-replicas for high-concurrency writes. 2. Task Queue: Replace FastAPI's in-memory BackgroundTasks with Celery or Temporal backed by Redis/RabbitMQ, distributing tasks across worker nodes. 3. Browser Pool: Browser instances consume ~150MB RAM each. We would manage a distributed headless browser pool (or use Browserless.io) with session reuse and rate-limiting proxies.

Q9: What security practices did you implement in this project?

1. Environment Secret Separation: API keys and webhook secrets are stored strictly in backend/.env, with a committed .env.example template. 2. CORS Restrictions: Configured FastAPI's CORSMiddleware to allow requests only from authorized frontend origins (http://localhost:5173). 3. Input Validation: All user search queries and preference strings are validated and bounded using Pydantic schemas. 4. SQL Parameterization: All database queries use parameterized SQL (? placeholders), eliminating SQL injection vulnerabilities.

Q10: What challenges did you face while developing this project, and how did you resolve them?

Two major engineering challenges stood out: Challenge 1: Headless Browser Environment Restrictions & Sandbox Permissions. When launching Chromium via Playwright in restricted sandbox environments, Browser Use defaulted to ~/.config/browseruse, triggering a PermissionError. I resolved this by configuring BROWSER_USE_CONFIG_DIR=./.browseruse in the application environment and creating a high-fidelity fallback runner. Challenge 2: Overwriting User Application States During Re-Crawl. Re-crawling was resetting 'Applied' listings to 'new'. I refactored the deduplication service to perform an URL lookup and preserve existing statuses.